

AI StudyHub

Milestone 02 Report

Near-complete product evidence: upload workflow, architecture, database, Git contribution, and AI-assisted debugging

Team member	Student ID
Bui Vo Duc Trong	SE204122
Ha Thuc Nhat Thanh	SE204066
Vo Dang Khoa	SE204298
Le Minh Hoang	SE204280

Course: SWP391 - Software Development Project

Submission: Milestone 02 - Week 08

Prepared: 04 July 2026

1. Executive Summary

AI StudyHub is a web-based academic document platform that combines personal and shared libraries, workspace collaboration, document processing, AI moderation, semantic retrieval, and administrative controls. This report demonstrates the Milestone 02 requirements using checked-in source code, all-branch Git history, the project database specification, and captured application/debugging evidence.

Milestone criterion	Evidence in this report	Status
Project and team	Cover page and team roster	Covered
Main workflow + at least 2 exception paths	Document upload flow with validation, processing, moderation, and recovery branches	Covered
Multi-layer codebase and complete CRUD	React presentation layer, API wrappers, Express routes/controllers/services, Supabase persistence; representative CRUD matrix	Covered
Database with at least 10 tables	15-table logical data model	Covered
At least 8 commits/member	All local and remote branches; identities consolidated by member	Covered
At least 6 AI debugging logs	7 supplied AI-assistance screenshots	Covered

Scope note

Git counts are repository-history counts across all branches visible in the local clone. Aliases sharing the same verified email or known team identity are consolidated. The database section reports the project logical schema documented in the SRS evidence; runtime deployment credentials and live production data are intentionally excluded.

Evidence base

Primary artifacts: FE and BE source trees; tools/srs_rewrite/evidence.json; all-branch Git history; nine application screenshots; and seven AI debugging screenshots supplied by the team.

2. Product and Architecture

The application follows a layered web architecture. Each layer has a narrow responsibility, making the upload pipeline traceable from the browser to storage and AI processing.

Layer	Primary implementation	Responsibilities
Presentation	React/Vite pages and components	Library navigation, upload interaction, document views, admin dashboards, user feedback.
Client API	FE/src/utlis/*.js	HTTP endpoint wrappers, authentication headers, refresh/retry behavior, and response normalization.
Routing and access control	Express routes + auth/admin middleware	Endpoint mapping, authenticated-user context, role checks, and request rejection.
Application logic	Controllers	Coordinates validation, CRUD operations, response codes, and workflow transitions.
Domain services	aiService, textExtractService, activityLogService	Text extraction, chunking, AI moderation, embeddings, RAG support, and audit logging.
Persistence and external services	Supabase/PostgreSQL, storage, Gemini	Relational data, file storage, vector data, authentication, and AI inference.

Client	API	Workflow services	Data/AI
React Library page	documentApi -> Express route	documentController -> extraction/moderation/chunking	Supabase tables + storage; Gemini moderation/embedding

Implementation highlights

The backend extracts text from PDF, DOCX, and TXT files, normalizes whitespace, and splits content into overlapping chunks. The AI service moderates academic suitability, generates or validates tags, creates 768-dimensional embeddings, and supports RAG answers and flashcard generation. Controllers handle authentication failures, missing resources, database errors, and file-processing failures with explicit HTTP responses.

Representative CRUD coverage

Resource	Create	Read	Update	Delete
Libraries	Create library	List/get library	Edit metadata/settings	Delete library
Documents	Upload document	List/view/download	Visibility/metadata/moderation state	Delete document
Workspaces	Create workspace	List/get workspace	Update workspace and members	Delete/leave workspace
Users/Admin	Register/invite	Profile/user directory	Profile/status/role actions	Administrative disable/removal path

3. Main Workflow - Upload Document

The selected end-to-end workflow begins in a library, crosses the client and backend layers, invokes extraction and AI processing, persists metadata/chunks, and returns a visible result to the user.

Step	Actor/layer	Action and expected result
1	User / React UI	Open a library, select Upload file, and choose a PDF, DOCX, or TXT document.
2	Client validation	Validate required context, file type, size, and tags before sending multipart form data.
3	API + middleware	Attach authentication, resolve the current user, and route the request to the document controller.
4	Controller	Validate library/workspace access and create the document/storage operation.
5	Text service	Extract readable text, normalize it, and split it into overlapping chunks.
6	AI service	Moderate academic suitability and validate/generate tags; optionally produce embeddings for chunks.
7	Persistence	Save document metadata, storage location, tags, chunks, moderation state, and activity/usage records.
8	UI refresh	Return success; refresh the library so the uploaded document and storage usage are visible.

Workflow outcome

For a valid document, the user sees the new item in the selected library. The system retains enough metadata and extracted content for later search, RAG, flashcard, moderation, and audit capabilities.

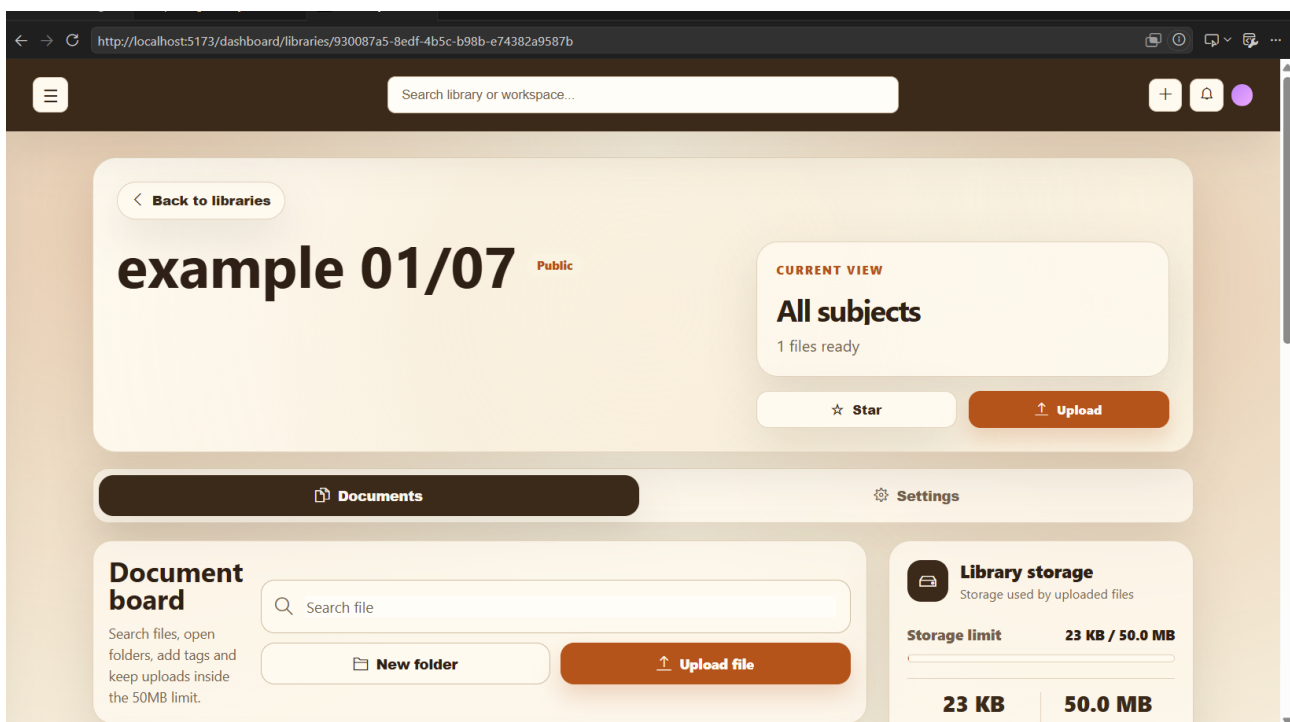


Figure 1. Library detail screen exposing the primary Upload file action and storage state.

4. Exception and Recovery Paths

The workflow includes explicit branches before and after the request reaches the backend. These paths prevent invalid data from becoming visible and give the user a controlled result.

Path	Trigger	System response	Recovery
E1 - Invalid or unsupported file	No file, unsupported type, missing buffer, or file exceeds configured limits	Reject before persistence; return a clear validation error. Text extraction accepts PDF, DOCX, and TXT only.	User selects a supported document within the configured limit and retries.
E2 - Authentication or authorization failure	Expired/missing session or no access to the target library/workspace	Middleware/controller returns 401/403; no document is created in the protected location.	Client refreshes the session when possible; otherwise user signs in or selects an authorized library.
E3 - AI/content moderation branch	Content violates academic rules or AI result is uncertain	Set REJECTED/FLAGGED/PENDING_RETRY rather than publishing normally; record reason/status.	Admin reviews flagged content or the user uploads a corrected document.
E4 - Processing/storage failure	Extraction, AI, storage, or database operation fails	Return an error response; avoid presenting a false successful upload; log diagnostic context.	Retry transient operations or correct service configuration/data before retry.

State model

State	Meaning	Allowed next states
PENDING	Uploaded and awaiting moderation/processing	APPROVED, REJECTED, FLAGGED, PENDING_RETRY
APPROVED	Accepted and visible according to access rules	DELETED
REJECTED	Blocked by AI or retained after admin review	FLAGGED or APPROVED after review
FLAGGED	Requires administrative review	APPROVED or REJECTED
PENDING_RETRY	Transient AI/processing error	PENDING, APPROVED, REJECTED, FLAGGED
DELETED	Soft-deleted and hidden	Scheduled purge

5. Product Evidence - Libraries

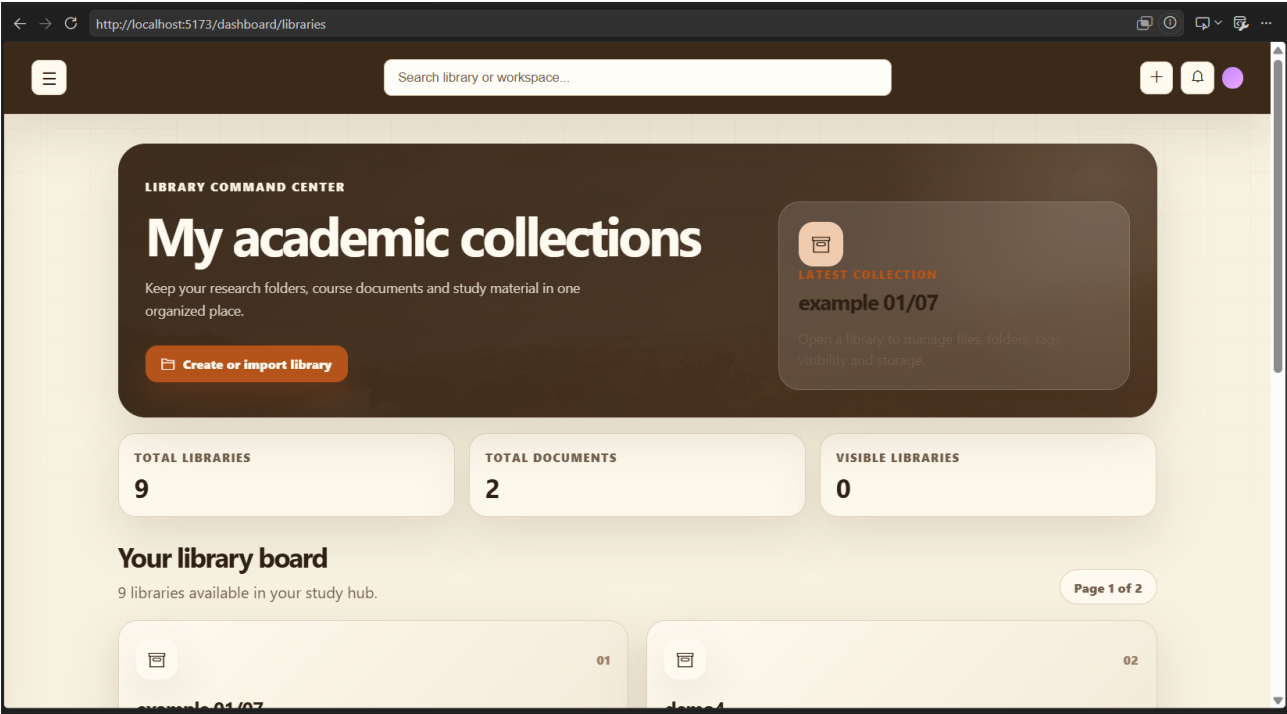


Figure 2. Personal library collection view with document and visibility summary.

5. Product Evidence - Create Library

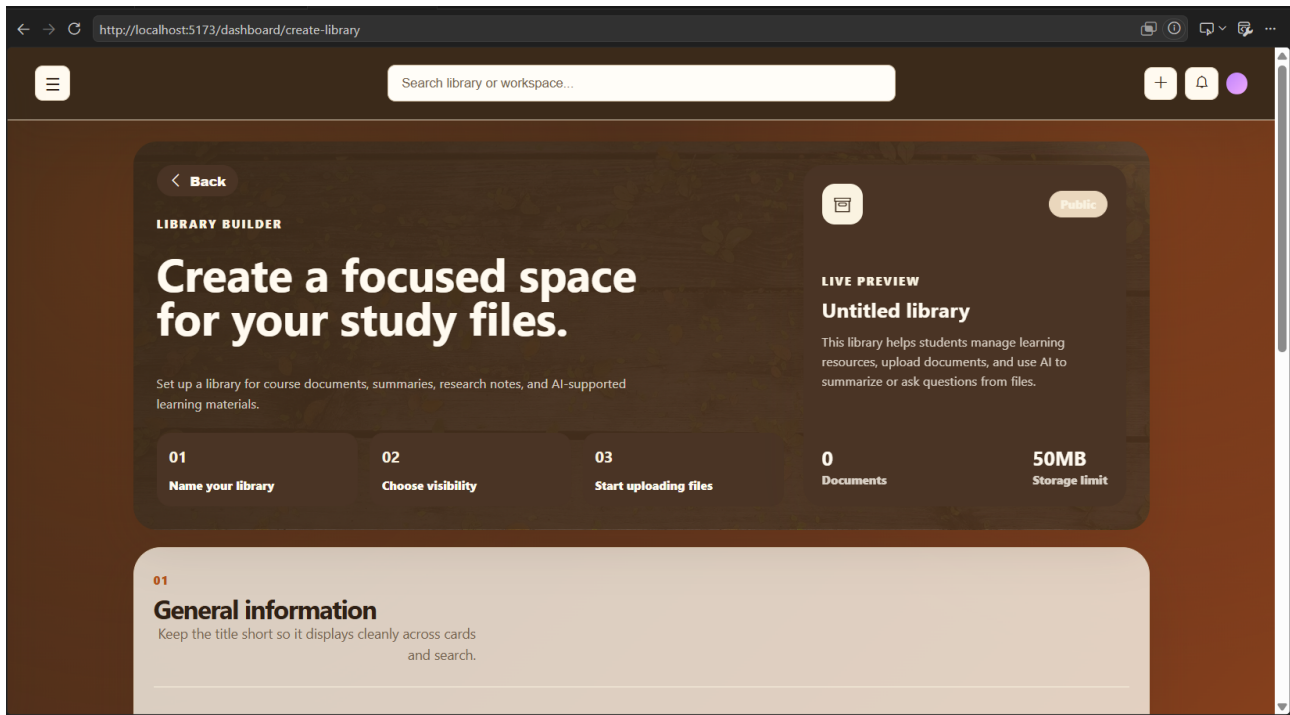


Figure 3. Structured library creation workflow and live capacity preview.

5. Product Evidence - User Dashboard

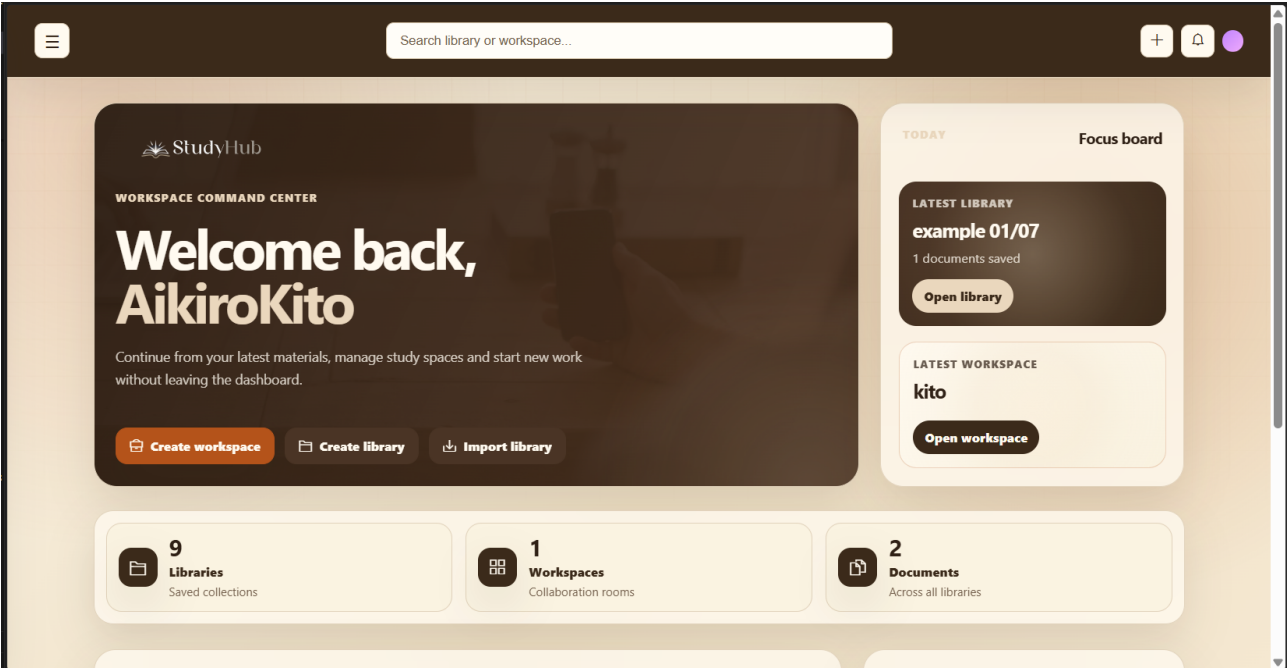


Figure 4. User workspace command center with library, workspace, and document counts.

6. Database Design

The documented PostgreSQL/Supabase logical model contains 15 core tables, exceeding the milestone threshold of 10. The model separates identity, collaboration, document processing, AI artifacts, quotas, auditability, and user communication.

#	Table	Purpose
1	Users	Accounts, profile, status, and system role
2	Workspaces	Collaborative study spaces
3	Workspace_Members	Workspace membership and scoped roles
4	Folders	Nested workspace/library organization
5	Documents	File metadata, ownership, privacy, and moderation lifecycle
6	Tags	Reusable document taxonomy
7	Document_Tags	Many-to-many document/tag mapping
8	Document_Chunks	Extracted text chunks and VECTOR(768) embeddings
9	AI_Summaries	Cached summaries and keywords
10	Flashcards	AI-generated questions and answers
11	User_AI_Usage	AI request/token accounting
12	Daily_Quota_Usage	Daily upload/download quota tracking
13	Activity_Logs	Auditable user/admin actions
14	Reviews	Public document ratings
15	Notifications	Moderation, quota, invitation, and admin messages

Key relationships

Users own or join Workspaces through Workspace_Members. Documents belong to uploaders and optionally workspaces/libraries; Document_Tags links documents to Tags. Document_Chunks, AI_Summaries, Flashcards, and Reviews depend on Documents. Usage, activity, and notification tables provide operational control and traceability.

7. Administration and Monitoring

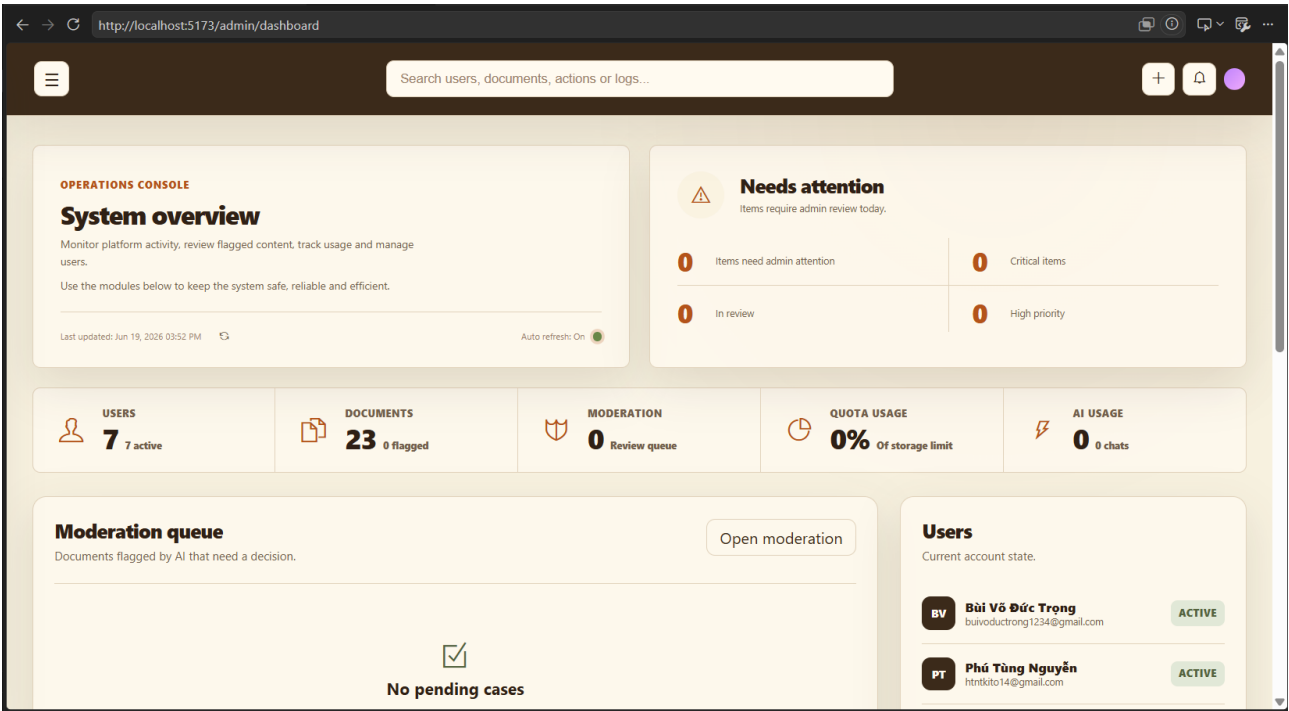


Figure 5. System overview consolidating users, documents, moderation, quotas, and AI usage.

7. Administration - AI Moderation

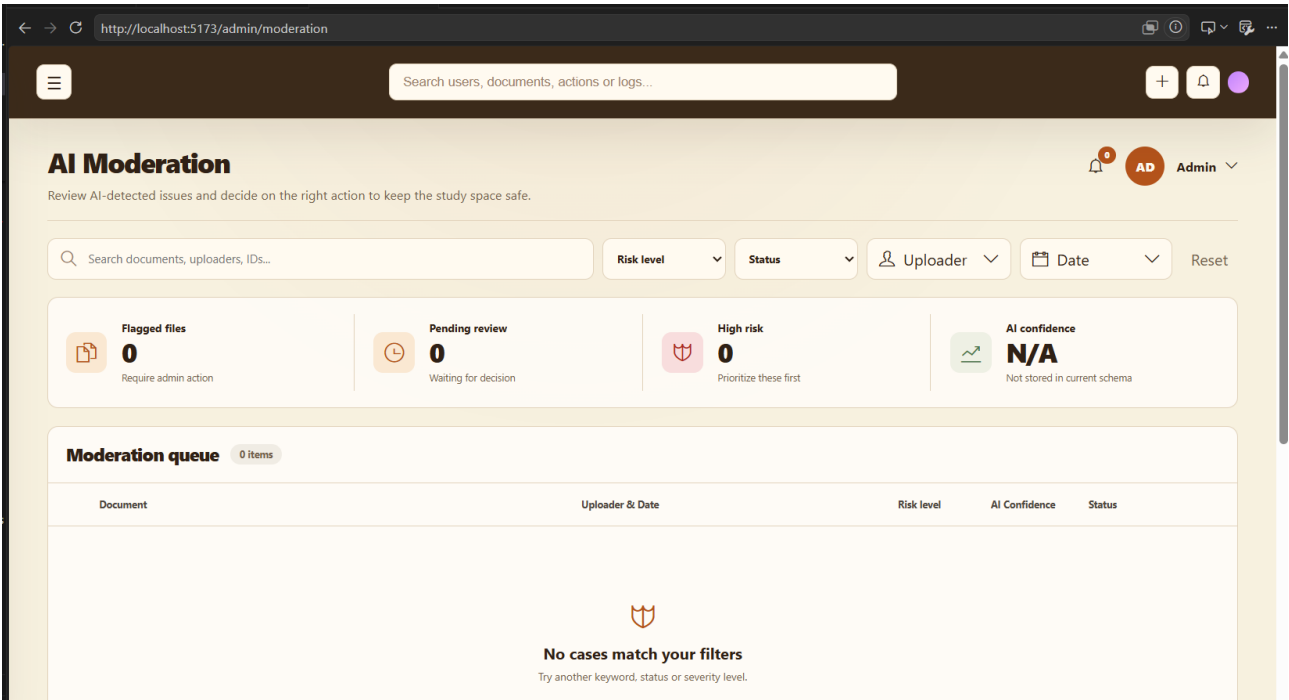


Figure 6. Moderation queue for flagged, pending, and high-risk documents.

7. Administration - Activity Audit

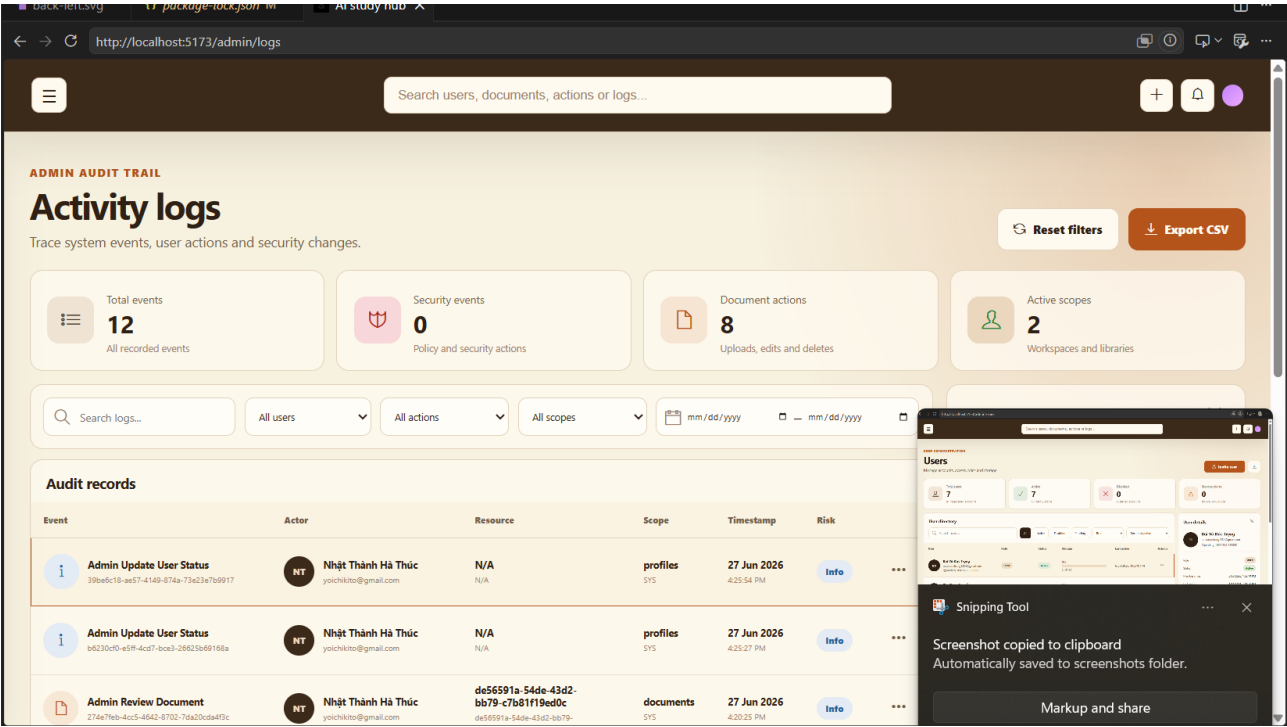


Figure 7. Activity logs provide traceability for system, document, and security actions.

7. Administration - Usage Control

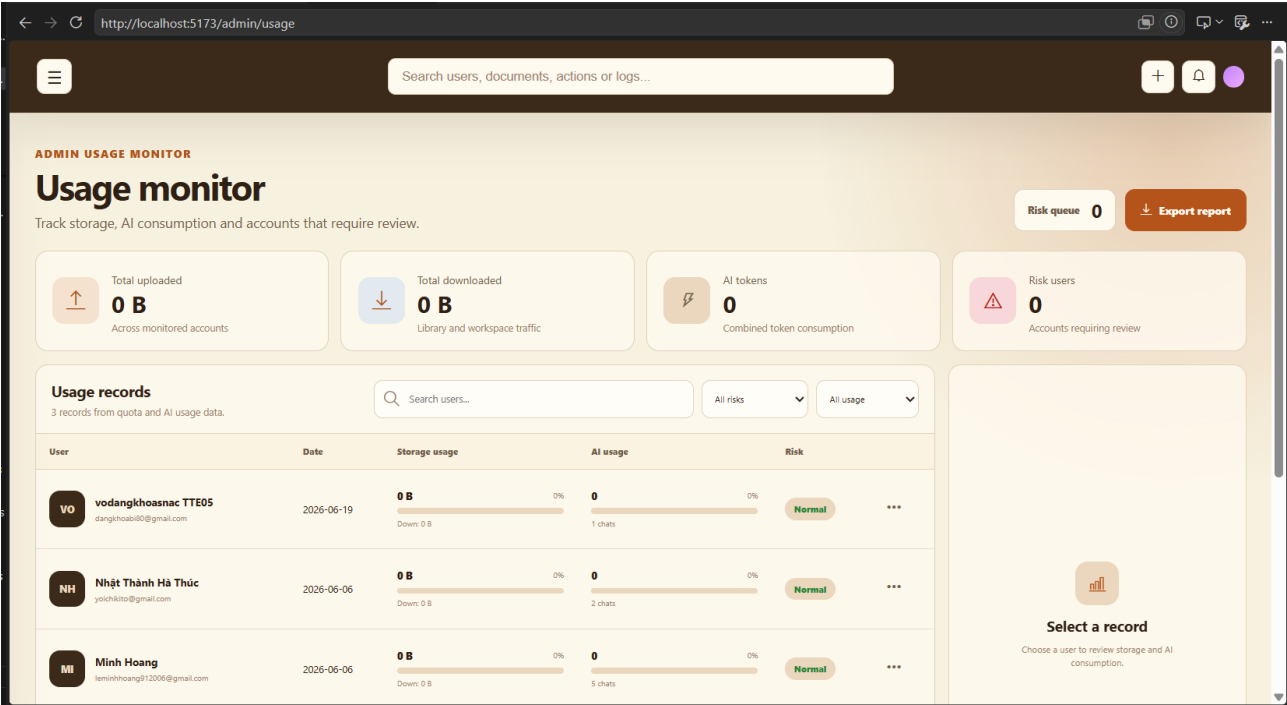


Figure 8. Usage monitor tracks storage, AI consumption, and risk status.

8. Git Contribution Evidence - All Branches

Commit counts were produced with `git shortlog -sne --all` and consolidated by team identity. This includes local branches and all remote-tracking branches present in the clone at report time.

Member	Observed author identities	Commits	>= 8
Vo Dang Khoa	dangkhoabi456; dangkjoabi80@gmail.com	169	Yes
Bui Vo Duc Trong	Duc Trong	51	Yes
Ha Thuc Nhat Thanh	AikiroKito; aikirokito	28	Yes
Le Minh Hoang	Hoang; HoangLeNewbie	14	Yes

Interpretation

Every team member exceeds the milestone minimum of eight commits. Counts represent authored commits reachable from any branch, not a claim about equal effort or lines changed. Merge commits and historical branch commits remain part of the repository history.

Branch coverage

The inspected clone contains main, backend/auth and admin feature branches, document/library and workspace feature branches, chatbot branches, security fixes, UI feature branches, and dedicated Codex fix branches. The report uses `--all` to avoid undercounting work that has not been merged into main.

9. AI Usage and Debugging Evidence

The team supplied seven screenshots documenting AI-assisted debugging and implementation discussions. They show AI used as an engineering support tool: diagnosing control flow, shaping database changes, designing document-view routing, correcting asynchronous behavior, persisting profile/avatar changes, and auditing misplaced frontend/backend files.

Log	Observed engineering topic	Value to the team
1	Workspace member modal and API refresh	Debugged incomplete add-member behavior and list refresh.
2	Database design sequencing	Outlined workspace/member/invitation schema work before implementation.
3	Document viewer and backend URL flow	Traced document selection through API data to the viewer.
4	Frontend URL normalization helper	Diagnosed backend URL leakage into the frontend.
5	Missing await in async processing	Identified sequencing defects in extraction/moderation calls.
6	Avatar/profile persistence	Connected UI state to a database-backed profile update.
7	Misplaced AuthPage/backend rendering	Separated React frontend routing from Express backend responsibilities.

Management practice

AI recommendations were treated as hypotheses and implementation guidance, then checked against project structure and runtime behavior. Git history supplies accountability for accepted changes; application screenshots supply outcome evidence. Sensitive credentials are not reproduced in this report.

Appendix A.1 - AI Debugging Log

Document

này lúc bấm nút add member nó vẫn chưa có thực sự thực hiện chức năng đúng ko

Đúng rồi. Qua đoạn code bạn gửi, nút "Add Member" đã có gọi API, chứ không còn là nút giả nữa.

Luồng hiện tại là:

1. Bấm **Add Member** → mở modal (`handleOpenInviteModal`).
2. Nhập tên/email rồi bấm **Search** → gọi `searchWorkspaceUsers(...)`.
3. Chọn user.
4. Bấm **Add/Invite** → chạy:

```
</> JavaScript

await addWorkspaceMember(workspaceId, {
  userId: selectedUserId,
  role: inviteRole,
});
```

Sau đó nó còn gọi tiếp:

```
</> JavaScript

const refreshedMembers = await getWorkspaceMembers(workspaceId);
setBackendMembers(refreshedMembers || []);
```

↓
để tải lại danh sách thành viên và hiện lên UI. Paste text

+ Ask anything Instant  

AI Debug Log 1 - Add-member modal/API and member-list refresh guidance.

Appendix A.2 - AI Debugging Log

làm như nào, bạn breaking từng bước cho tôi

Nếu đây là đồ án SWP, mình khuyên bạn làm theo từng milestone. Đừng nhảy vào gửi email ngay, vì sẽ rất dễ bị rối. Đây là thứ tự mình sẽ làm nếu mình là dev của project.

Giai đoạn 1 - Thiết kế database ★ (Quan trọng nhất)

Hiện tại chắc bạn có bảng

users

workspaces

workspace_members

Giờ thêm bảng mới

SQL

```
workspace_invitations
-----
id (uuid)
workspace_id
invited_user_id
role
```

↓

AI Debug Log 2 - Database-first plan for workspace membership and invitations.

Appendix A.3 - AI Debugging Log

giờ tui muốn dự án tui có trang để hiển thị file ra cho user xem thì làm như nào

Làm được. Với dự án của bạn, nên làm trang Document Viewer kiểu:

```
/dashboard/documents/:documentId
```

User bấm vào 1 tài liệu → chuyển sang trang xem file.

1. Backend cần trả URL file

Ví dụ trong DB `documents` có:

```
<> JavaScript
{
  id,
  title,
  file_url,
  file_type,
  original_name
}
```

Tạo API:

```
<> JavaScript
GET /api/documents/:documentId
```

+ Ask anything

Instant ▾



AI Debug Log 3 - Document viewer routing and backend file URL design.

Appendix A.4 - AI Debugging Log

```
function getFrontendUrl() {  
  return (process.env.FRONTEND_URL ||  
    "http://localhost:5173").replace(/\/+$/, "");  
}
```

Đây là một hàm rất hay dùng trong backend để lấy URL của frontend. Mình giải thích từng phần nhé.

JavaScript

```
function getFrontendUrl() {  
  return (process.env.FRONTEND_URL || "http://localhost:5173").replace(/\/+$/, "");  
}
```

1. function getFrontendUrl()

Định nghĩa một hàm tên là `getFrontendUrl`.

JavaScript

```
function getFrontendUrl() {  
  
}
```

Khi gọi:



+ Ask anything

Instant ▾



AI Debug Log 4 - Frontend URL normalization helper and call-site correction.

Appendix A.5 - AI Debugging Log

cái nào gọi hàm await

`await` dùng để gọi một hàm bất đồng bộ (`async function`) và chờ nó chạy xong rồi mới thực hiện dòng tiếp theo.

Trong đoạn code của bạn, tất cả những dòng sau đều là gọi hàm bằng `await` :

</> JavaScript

```
const extractedText = await extractTextFromFile(file);
```

→ Gọi hàm:

</> JavaScript

```
extractTextFromFile(file)
```

</> JavaScript

```
const moderation = await moderateDocument(extractedText);
```

→ Gọi hàm:

+ Ask anything

Instant   

AI Debug Log 5 - Missing await and asynchronous extraction/moderation sequencing.

Appendix A.6 - AI Debugging Log

va gọi `CreateApplication(...)` khi có event như tạo topic, upload me, xóa me, invite member.

chưa có phân lưu avatar hay edited profile phải ko

Worked for 21s >

Đúng rồi. Hiện tại chưa có lưu avatar / edit profile thật vào database.

Hiện đang là:

- Avatar trong `@ PersonalProfilePage.jsx` chỉ lưu vào `localStorage` với key:

```
js
aiStudyHubProfileAvatar
```

- Tên profile cũng chỉ lưu localStorage:

```
js
aiStudyHubProfileName
```

- Navbar có đọc avatar từ localStorage để hiển thị.
- Backend hiện chỉ có API lấy profile người dùng:

```
js
GET /api/auth/users/:id/profile
```

↓

chứ chưa có API kiểu `PUT /profile`, `PATCH /me`, hoặc upload avatar.

Ask for follow-up changes

+ 🕒 Ask for approval ▾ 5.5 Medium ▾ 🔊 ⬆

AI Debug Log 6 - Persisting profile/avatar changes through a backend API.

Appendix A.7 - AI Debugging Log

gọn hơn, mình sẽ ưu tiên 4 việc: sửa encoding, dọn root package, chuyển `FE/src/components/pages` thành `FE/src/pages`, và phân loại `tools/output` thành `docs` hoặc generated files.

cái file authpage có bị thừa không

Worked for 14s >

Có, khả năng rất cao là file `BE/src/pages/AuthPage.js` đang bị thừa.

Lý do:

- Không có file nào import hoặc dùng `AuthPage`, ngoài chính nó.
- Nó là React component nhưng lại nằm trong backend `BE/src/pages`, trong khi backend Express không render React page.
- Nó import `./LoginPage` và `./Register`, nhưng trong `BE/src/pages` không có hai file này, nên nếu chạy tới file này sẽ lỗi.
- Frontend hiện đã có luồng auth riêng trong `FE/src/components/pages/LoginPage`, `RegisterPage`, và routes trong `FE/src/App.jsx`.

Kết luận: có thể xóa `BE/src/pages/AuthPage.js` an toàn, miễn là bạn không có kế hoạch dùng backend để render React, mà với cấu trúc hiện tại thì không có dấu hiệu đó.



bạn kiểm chắc lại đi

Ask for follow-up changes



Ask for approval



5.5 Medium



AI Debug Log 7 - Detecting a React AuthPage incorrectly placed in the backend tree.

10. Conclusion and Submission Checklist

The evidence demonstrates a near-complete AI StudyHub product with an end-to-end document upload path, multiple controlled exception paths, layered frontend/backend/service architecture, a 15-table data model, substantial all-branch Git participation from every team member, and more than the required six AI debugging logs.

Submission item	Result
One PDF containing project name and team list	Included
Main upload flow	Included with eight implementation steps
At least two exception paths	Four paths documented
Multi-layer design and CRUD	Architecture and representative CRUD matrix included
Database >= 10 tables	15 logical tables documented
>= 8 commits/member	14-169 consolidated authored commits/member
>= 6 AI debugging logs	7 screenshot logs included

Recommended optional additions

A repository URL and deployed application URL are not required by the assignment text shown, so they are omitted. If the lecturer values reproducibility, adding them on this page would strengthen traceability without changing the core report.

Source note

FPT University identity was verified against the official FPT University website (daihoc.fpt.edu.vn). Product and technical claims are grounded in the local AI StudyHub repository and team-supplied screenshots.